

Livello di micro-architettura

CPU E CICLO D'ISTRUZIONE

Livelli di un sistema di elaborazione



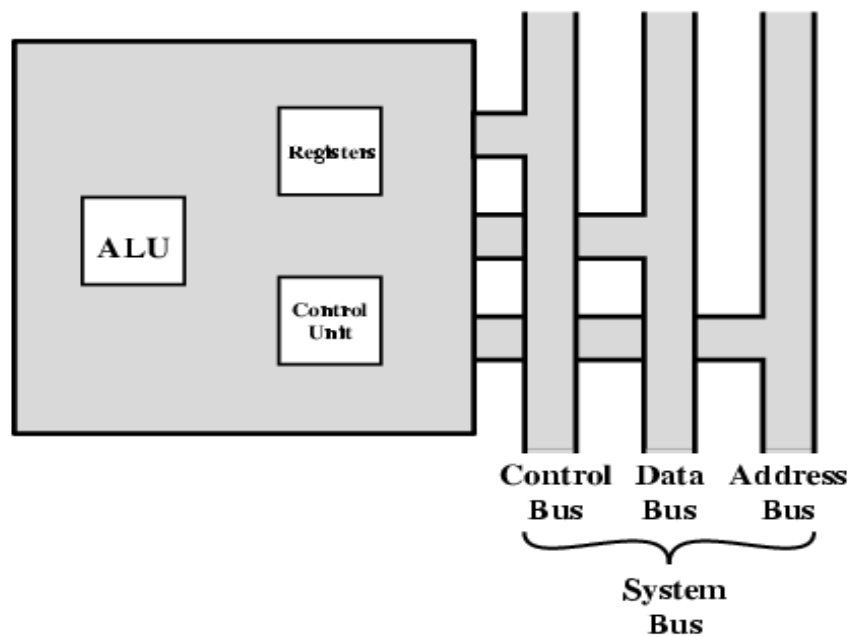
Definizione di microarchitettura

- Per microarchitettura s'intende l'**organizzazione dei componenti interni** alla CPU
- Essa è trasparente al programmatore
 - Il programmatore utilizza l'elaboratore attraverso le istruzioni
 - Le istruzioni sono interpretate a livello di microarchitettura
 - Ciclo di esecuzione delle istruzioni

Compiti della CPU

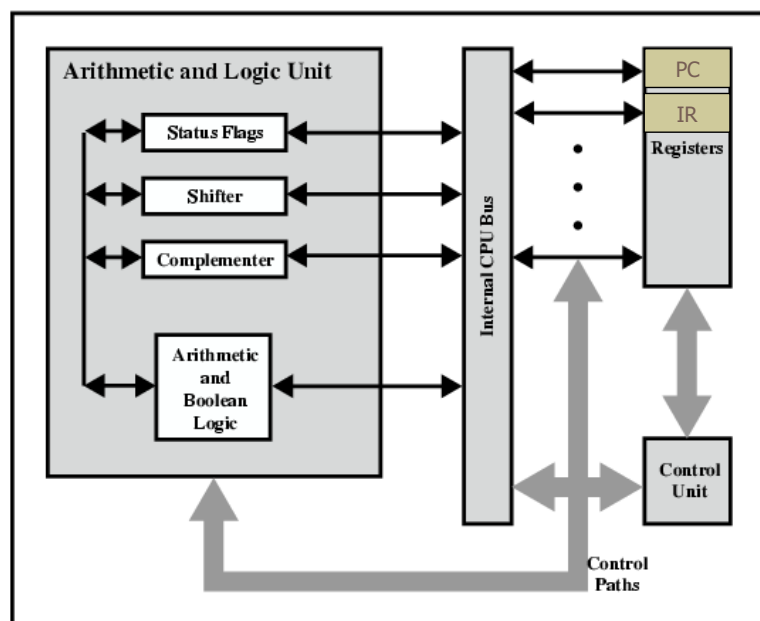
- La CPU deve:
 1. Prelevare un'istruzione dalla memoria
 2. Interpretarla
 3. Prelevare eventuali operandi dalla memoria
 4. Eseguire l'istruzione
 5. Scrivere gli eventuali risultati in memoria
- La CPU esegue ciclicamente questi passi → Ciclo d'istruzione

CPU con bus di sistema



5

Struttura interna della CPU



6

Bus

- Il bus è l'hardware per il trasferimento di informazioni tra i dispositivi:
 - un insieme di linee (tracce di rame) su cui viaggiano le informazioni sotto forma di segnali digitali
 - su ogni linea viaggia un singolo bit
- Le informazioni trasferite sono
 - Indirizzi
 - Dati
 - Segnali di controllo

Componenti del Bus

- **Bus indirizzi**
 - Funzione: selezione della unità con cui stabilire la comunicazione
 - trasmette l'indirizzo della cella di memoria da leggere o scrivere, o il numero di registro di un dispositivo di I/O
 - Caratteristica: ha origine dall'unità di calcolo indirizzi della CPU ed è **unidirezionale**
 - La sua larghezza determina la dimensione dello **spazio di indirizzi** (numero di locazioni di memoria indirizzabili) e il numero di unità periferiche selezionabili

Componenti del Bus

- **Bus dati**

- Funzione: trasmissione e ricezione di dati
- Caratteristica: **bidirezionale**
- La sua larghezza stabilisce la **grandezza del dato** (ampiezza in bit) trasferibile in un ciclo di clock
 - Collo di bottiglia dell'elaboratore di Von Neumann



10

Componenti del Bus

- **Linee di controllo**

- Funzione: trasmissione di segnali di controllo che specificano informazioni eterogenee
 - Tipologia di trasferimento dei dati (read/write)
 - Richieste di autorizzazioni (request)
 - Segnali di notifica (acknowledgment e granting)
 - Interruzioni esterne (interrupt)
 - ...
- Caratteristica: linee **unidirezionali** e **bidirezionali**
- Il numero di linee non è prefissato e può variare in base a vincoli architetturali
 - Segnali alternativi (WR come negazione di RD)
 - Distinzione fra indirizzamento memoria/periferica (isolated I/O)
 - ...

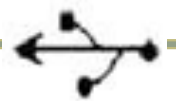
11

Tipi di bus - Trasmissione

- **Bus paralleli:** trasmissione parallela
 - i bit che costituiscono l'informazione sono trasmessi in parallelo attraverso fili separati
 - Esempi
 - ISA (Industry Standard Architecture, 1981)
 - SCSI (Small Computer System Interface, 1986)
 - PCI (Peripheral Component Interconnect, 1992)
 - PCI Express (2004)
- **Bus seriali:** trasmissione sequenziale
 - i bit che costituiscono l'informazione sono trasmessi uno alla volta su un solo "filo"
 - Esempi
 - USB (Universal Serial Bus, 1996)

12

Bus USB (1996, 2000, 2008, ...)



- **USB** (Universal Serial Bus)
 - lo standard 1.0 (1996) ha una velocità di trasmissione di solo **1,5 Mbit/s**
 - nella versione 2.0 (2000) raggiunge una velocità di trasmissione di **480 Mbit/sec**
 - nella versione 3.0 (2008) raggiunge una velocità di trasmissione di **4,8 Gbit/sec**
 - nella versione 4 (2021) raggiunge una velocità di trasmissione di **40 Gbit/sec (5GB/s)**
- Usato per quasi tutte le periferiche (tastiere, scanner, mouse, stampanti)
 - soprattutto per quelle multimediali (fotocamere digitali, webcam, ...)

13

Bus USB

- Progetto nato dall'accordo di centinaia di società di produttori
- Obiettivi che hanno dato il via al progetto:
 - facilità d'uso per l'utente
 - unico cavo per la connessione di dispositivi diversi
 - alimentazione per i dispositivi portata dal cavo
 - fino a 127 dispositivi collegabili al calcolatore
 - supporto per dispositivi funzionanti in tempo reale (telefono, audio, ecc)
 - installazione dei dispositivi "a caldo"
 - bassi costi di produzione

Registri

- La CPU deve disporre di spazio per la memorizzazione temporanea dei dati
 - Registri
- Il numero e la funzione dei registri dipende dalla CPU
- Livello più alto della gerarchia di memoria
- Un registro è un insieme di n memorie di 1 bit (flip-flop) che possono essere lette o scritte in parallelo
 - È usato per memorizzare una word di n bit

Registri

- **Registri visibili al programmatore**
 - Il programmatore in linguaggio macchina o Assembly può utilizzare tali registri in modo da minimizzare i riferimenti alla memoria
- **Registri di controllo e di stato**
 - Utilizzati dall'unità di controllo per controllare le operazioni della CPU
 - Utilizzati da programmi del sistema operativo per controllare l'esecuzione dei programmi

Registri di controllo

- **PC** (Program Counter)
 - contiene l'indirizzo della prossima istruzione da eseguire
- **IR** (Instruction Register)
 - contiene l'istruzione che deve essere eseguita
- **MAR** (Memory Address Register)
 - contiene l'indirizzo della locazione di memoria usata per lettura o scrittura di un dato
- **MDR** (Memory Data Register) o **MBR** (Memory Buffer Register)
 - contiene il dato che è stato letto o che deve essere scritto nella locazione indirizzata dal MAR
- **PSW** (Program Status Word): parola di stato di programma
 - Contiene i **flag di stato**, bit impostati dalla CPU dopo un'operazione
 - Ogni bit ha un significato specifico
 - Accessibile parzialmente dal programmatore

Registri visibili al programmatore

- **Registri generali o general-purpose**
 - possono avere diverse funzionalità, stabilite di volta in volta dal programmatore
- **Registri per i dati**
 - Possono essere utilizzati solo per contenere dati
- **Registri per gli indirizzi**
 - Sono dedicati a particolari modalità di indirizzamento
 - Es. puntatori a segmenti, puntatori a stack, registri indice

18

Esempio di organizzazione dei registri

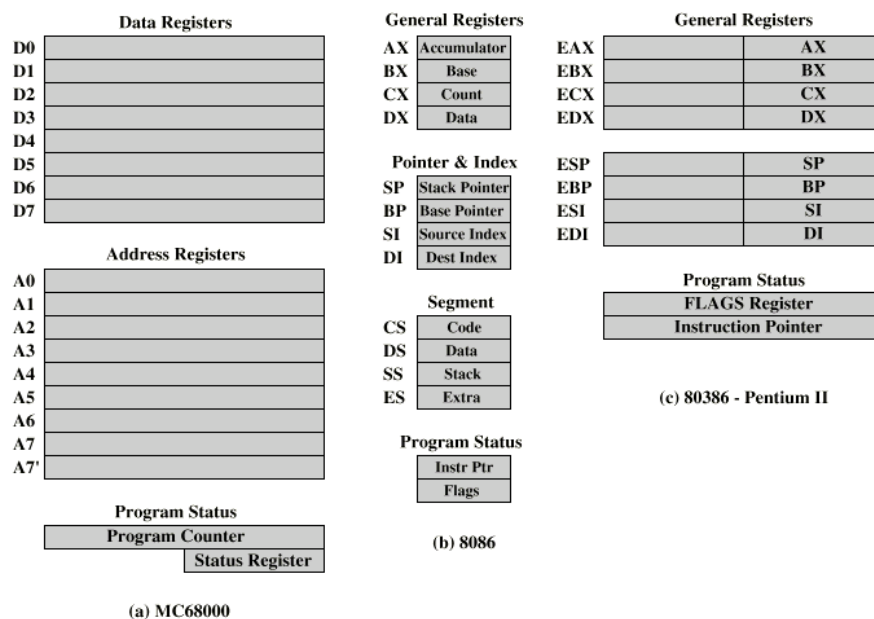


Figure 11.3 Example Microprocessor Register Organizations

20

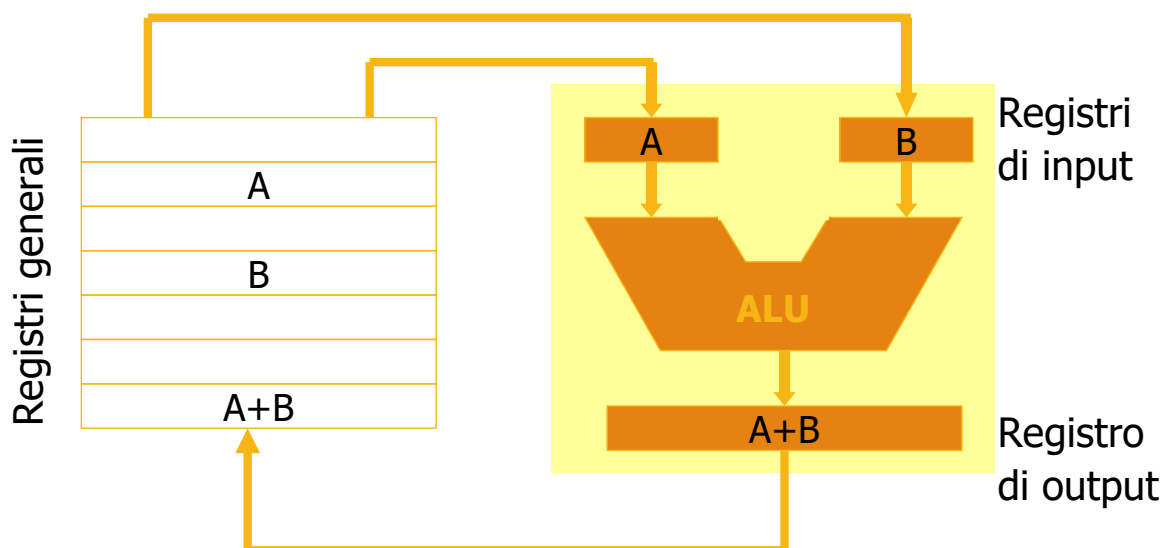
Unità Aritmetico-logica

- Dispositivo in grado di eseguire operazioni aritmetiche e logiche sui dati memorizzati nei registri



21

Il Data Path



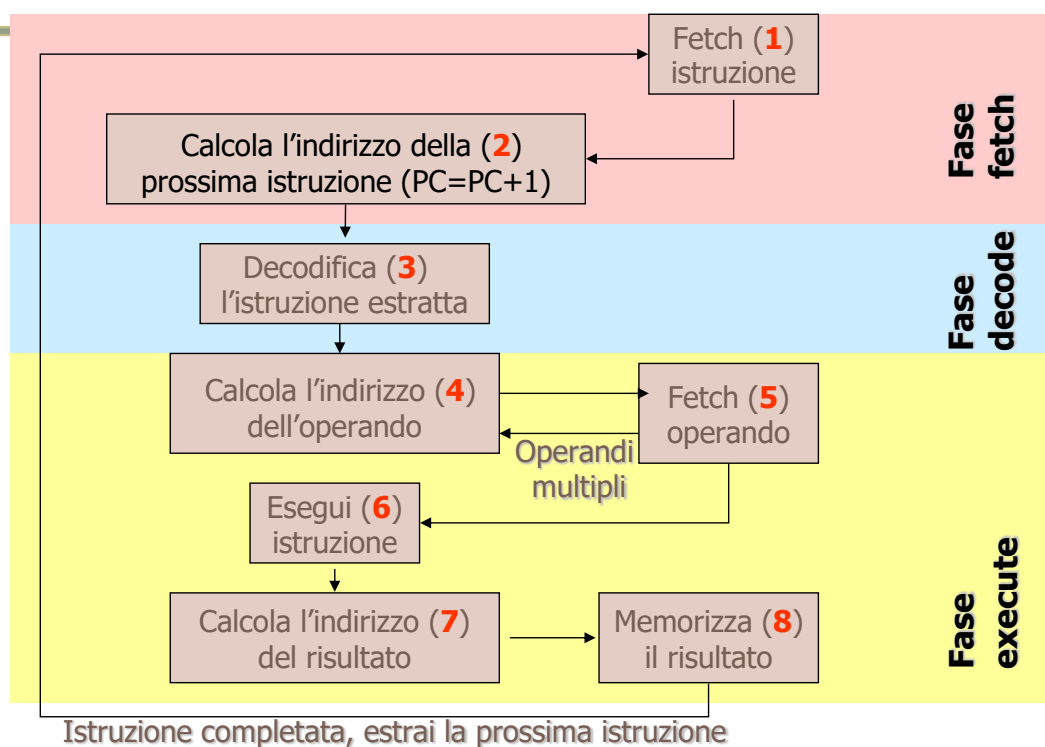
22

Unità di controllo

- Dispositivo in grado di “interpretare” ogni istruzione codificata in linguaggio macchina tramite una sequenza di **azioni elementari** direttamente eseguite in hardware
- Compito fondamentale dell’unità di controllo è quello di generare nella giusta sequenza i **segnali di controllo** necessari a controllare la sequenza di operazioni del ciclo di istruzione

23

Ciclo d’istruzione



24

Ciclo di esecuzione delle istruzioni

- Suddivisione dell'esecuzione di una istruzione in micro-operazioni
 - **Fase fetch:**
 - Il valore di PC viene trasferito in MAR
 - Il valore di MAR viene trasferito sul bus indirizzi esterno
 - L'indirizzo sul bus indirizzi raggiunge la memoria
 - Si imposta il bus di controllo per una richiesta di lettura da memoria
 - Il segnale di controllo raggiunge la memoria
 - Il risultato della richiesta viene posto sul bus dati esterno
 - Si trasferisce in MBR il contenuto del bus dati esterno
 - Si trasferisce in IR il contenuto di MBR
 - Si incrementa PC

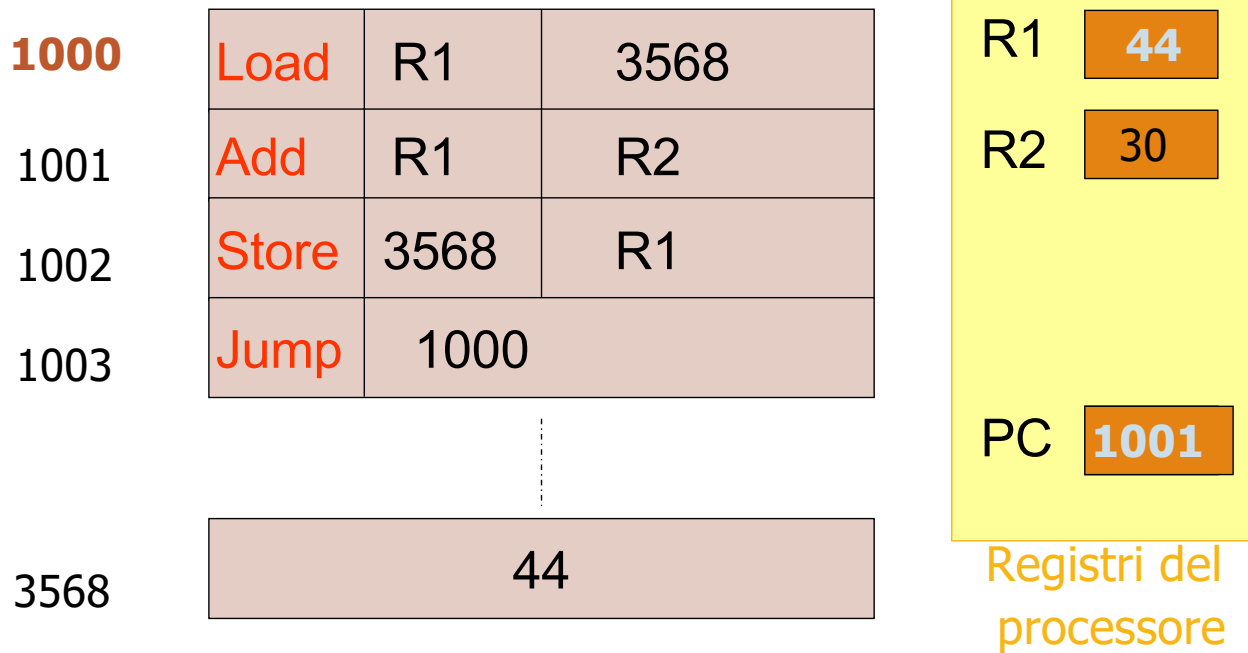
25

Ciclo di esecuzione delle istruzioni

- **Fase decode:**
 - Il valore di IR viene decodificato, individuando l'operazione (codice operativo) da eseguire e gli eventuali operandi
 - Alla stringa di bit dell'istruzione corrispondono segnali di controllo
 - Se l'istruzione richiede la lettura degli operandi, si valorizza il registro MAR per l'accesso alla memoria, in modo che in MBR sarà presente il valore dell'operando
- **Fase execute:**
 - I segnali di controllo ottenuti dalla fase di decodifica sono utilizzati per abilitare i trasferimenti tra registri e ALU, nonché alla selezione dell'operazione che l'ALU deve eseguire
 - Se l'istruzione prevede la scrittura dei risultati in memoria, si trasferiscono i risultati in MBR e si valorizza MAR affinché contenga l'indirizzo dove memorizzare i risultati

26

Esempio (1)



33

Esecuzione di (Load R1, 3568)

- PC = 1000
- MAR = 1000
- MDR = (Load R1, 3568)
- IR = (Load R1, 3568)
- PC = 1001
- Op_code = Load
- MAR = 3568
- MDR = 44
- R1 = 44

Lettura
dell'istruzione
all'indirizzo 1000

34

Esempio (2)

1000	Load	R1	3568
1001	Add	R1	R2
1002	Store	3568	R1
1003	Jump	1000	

3568

44

R1 74

R2 30

PC 1002

Registri del
processore

35

Esecuzione di (Add R1, R2)

- R2 = 30
- R1 = 44
- PC = 1001
- MAR = 1001
- MDR = (Add R1, R2)
- IR = (Add R1, R2)
- PC = 1002
- Op_code = Add
- R1 = 44+30 = 74

Lettura
dell'istruzione
all'indirizzo 1001

/* Viene attivata la ALU */

36

Esempio (3)

1000	Load	R1	3568
1001	Add	R1	R2
1002	Store	3568	R1
1003	Jump	1000	

3568

74

R1 74

R2 30

PC 1003

Registri del
processore

37

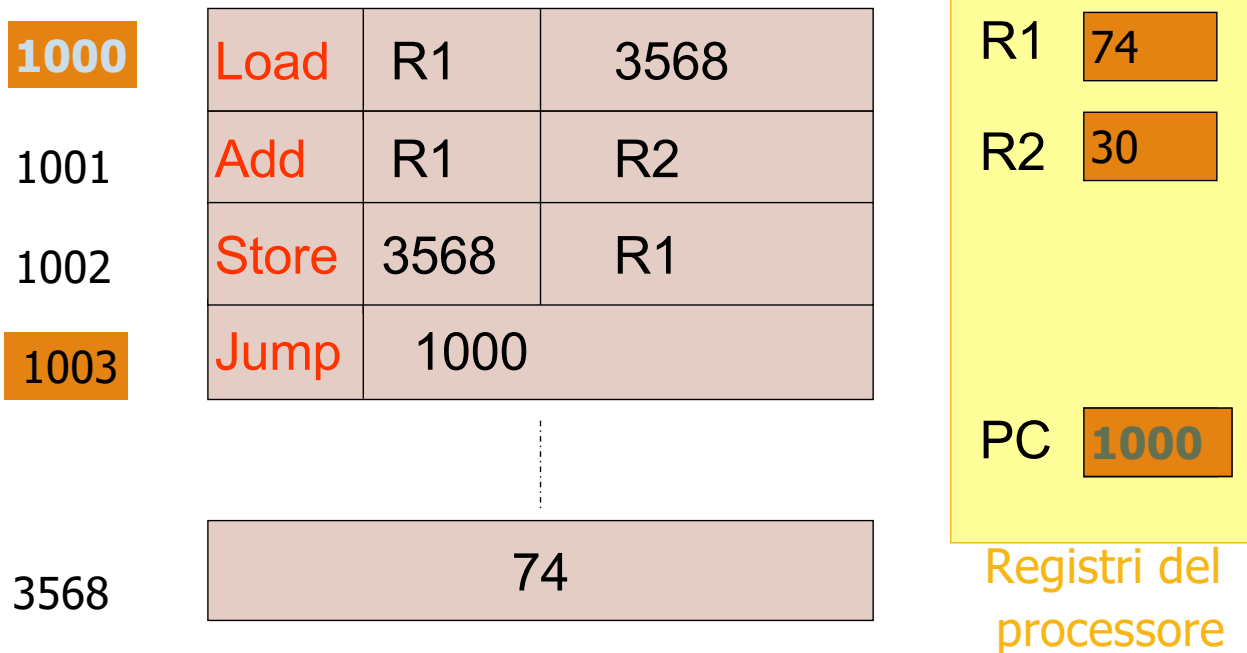
Esecuzione di (Store 3568, R1)

- PC = 1002
- MAR = 1002
- MDR = (Store R1, 3568)
- IR = (Store R1, 3568)
- PC = 1003
- Op_code = Store
- MAR = 3568
- MDR = 74
- Nella locazione 3568 viene scritto il numero 74

Lettura
dell'istruzione
all'indirizzo 1002

38

Esempio (4)



39

Esecuzione di (Jump 1000)

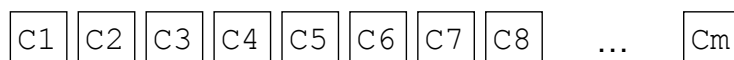
- PC = 1003
- MAR = 1003
- MDR = (Jump 1000)
- IR = (Jump 1000)
- PC = 1004
- Op_code = Jump
- PC = 1000

Lettura
dell'istruzione
all'indirizzo 1003

40

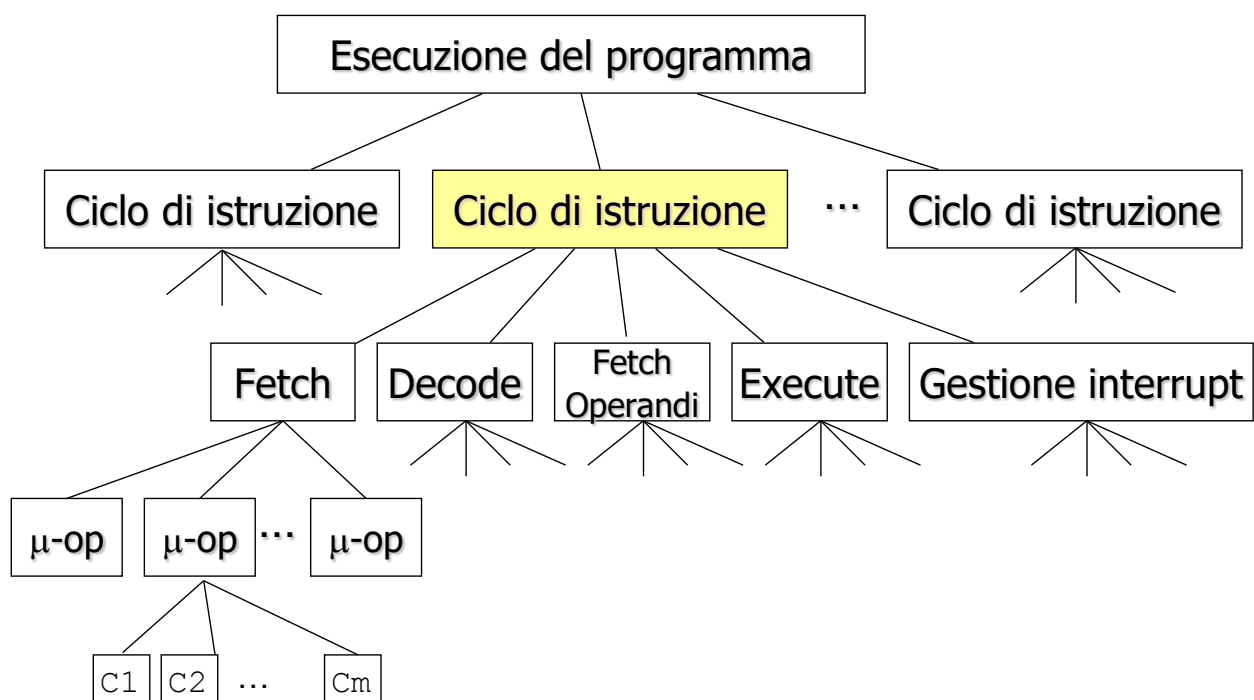
Ciclo d'istruzione

- Ogni fase del ciclo di istruzione è a sua volta costituita da una sequenza di operazioni elementari, dette **micro-operazioni**
- Ogni micro-operazione può essere
 - un trasferimento di dati da un registro a un altro
 - un trasferimento di dati da/a un registro al/dal bus di sistema
 - una operazione aritmetica o logica
- Ogni micro-operazione è realizzata attraverso un insieme di **segnali di controllo**
- I segnali di controllo sono l'output dell'unità di controllo e si possono rappresentare come vettori di bit:



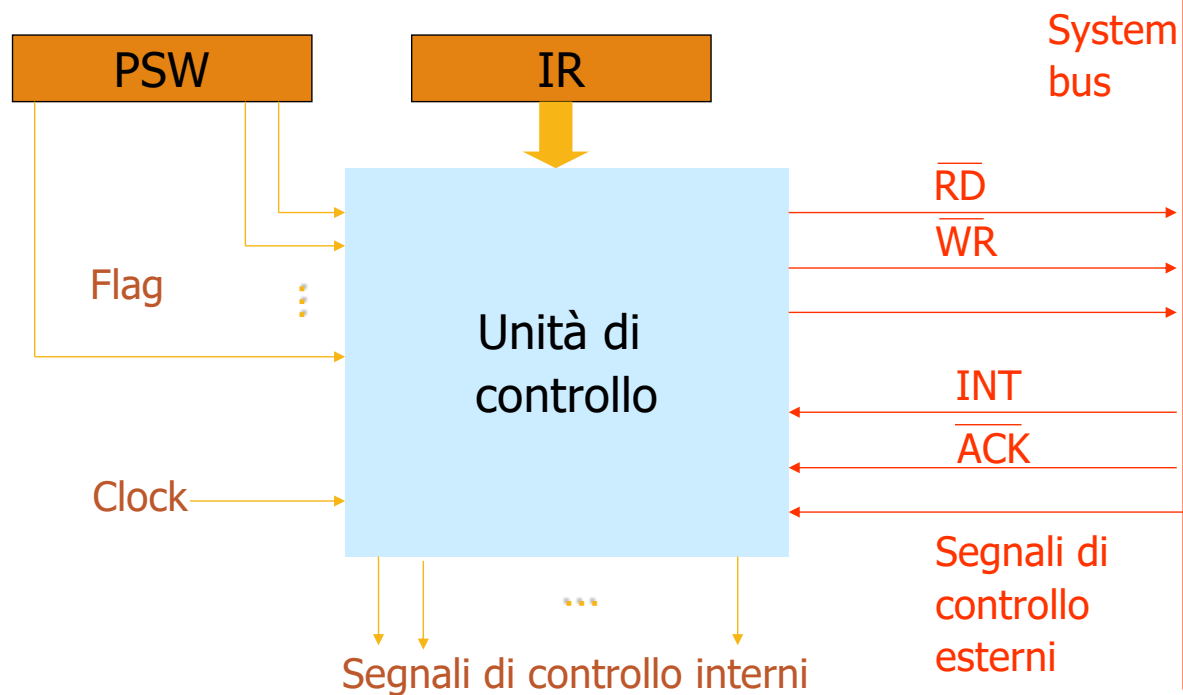
41

Esecuzione di un programma



42

Componente di controllo della CU



44

Sequenza di micro-operazioni del ciclo d'istruzione

$t_1: MAR \leftarrow PC$

$t_2: MDR \leftarrow Mem[MAR]$

$t_3: IR \leftarrow MDR$

$PC \leftarrow PC + L$

Fetch istruzione

$t_1: MAR \leftarrow IR.address$

$t_2: MDR \leftarrow Mem[MAR]$

Fetch operando

Execute

$t_1: MDR \leftarrow PC$

$t_2: MAR \leftarrow Save_address$

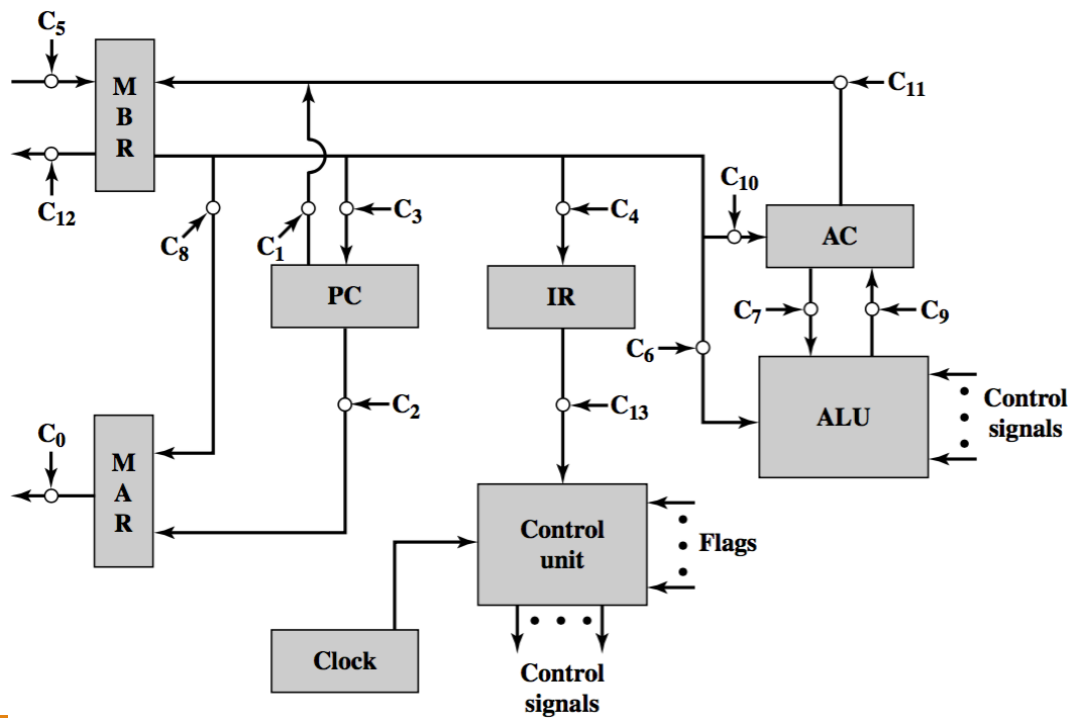
$PC \leftarrow ISR_address$

$t_3: Mem[MAR] \leftarrow MDR$

Gestione interrupt

45

Esempio di microarchitettura



48

Segnali di controllo per la fase di fetch

- L'unità di controllo coordina lo svolgimento delle micro-operazioni della fase di fetch attivando opportuni segnali di controllo
 - **MAR ← PC**
 - un segnale (interno) che apre il collegamento tra PC e MAR
 - **MDR ← Mem[MAR]**
 - un segnale di lettura dalla memoria (esterno in uscita)
 - un segnale (interno) che apre il collegamento tra MDR e linee del bus di dati
 - **IR ← MDR**
 - un segnale (interno) che apre il collegamento tra MDR e IR
 - **PC ← PC + L**
 - segnali di controllo (interni) per incrementare PC

49

Sequenza di micro-operazioni del ciclo d'istruzione

Esempio: ADD ACC, [100]

Fetch	1.	MAR $\leftarrow$ PC	C2
	2.	MBR $\leftarrow$ memory[MAR] PC $\leftarrow$ PC + L	C0,C5
	3.	IR $\leftarrow$ MBR	C4
Operand fetch	1.	MAR $\leftarrow$ 100	C8
	2.	MBR $\leftarrow$ memory[MAR]	C0,C5
Execute	1.	ALU $\leftarrow$ MBR, ACC	C6, C7
	2.	ALU : +	
	3.	ACC $\leftarrow$ ALU	C9